

UNIVERSIDAD NACIONAL DE COLOMBIA
Facultad de Ingeniería
Departamento de Ingeniería de Sistemas e Industrial

Programación Dinámica

Matemáticas Discretas I

Diego Alejandro Robayo Córdoba

Docente: Jhoan Sebastián Tenjo García

Bogotá, Colombia
22 de julio de 2026

1. Introducción

En este documentose describe el funcionamiento y las características algoritmicas de diez problemas de programación dinámica, donde se incluye Criptografía, grafos, álgebra de Boole, Shannon.

2. Cifrado César: romper un mensaje antiguo

El cifrado de César consiste en modelar el alfabeto como un conjunto de índices de 0 a 25, donde cada letra se reemplaza por otra que se encuentra un número fijo de posiciones más adelante en el alfabeto. Por ejemplo, si el desplazamiento es 3, la letra A se reemplaza por D, B por E, y así sucesivamente. Esta expresión se expresa mediante aritmética modular de la forma

$$(x + k) \text{ mód } 26$$

donde x es el índice de la letra original, k es el desplazamiento y 26 es el número de letras en el alfabeto. De esta forma, si el índice se pasa de 26, con el modulo se vuelve al inicio del alfabeto para recorrerlo nuevamente.

En cuanto el descifrado, se hace la operación inversa, es decir, se resta el desplazamiento. Por ejemplo, si el desplazamiento es 3, luego del descifrado la letra D se reemplaza por A, E por B, y así sucesivamente. Esta expresión se expresa mediante aritmética modular de la forma

$$((x + k) - k) \text{ mód } 26 = x$$

Debido al espacio de claves tan pequeño que tiene el alfabeto, el cifrado de César es muy fácil de romper en un ataque de fuerza bruta. Por esto es posible probar todas las claves posibles hasta encontrar un texto legible. Si tomamos como ejemplo el mensaje cifrado "KROD XQDO", podemos probar todas las claves posibles y obtener el siguiente resultado:

$$k = 0 : KROD \quad XQDO$$

$$k = 1 : JQNC \quad WPCN$$

$$k = 2 : IPMB \quad VOBM$$

$$k = 3 : HOLA \quad UNAL$$

3. RSA de juguete: llaves, cifrado y descifrado

Primero se seleccionan dos números primos p y q . A partir de ellos se calcula $n = p \times q$. Luego se calcula la función ϕ de Euler del numero n . Como n es el producto de dos primos, entonces

$$\phi(n) = (p - 1)(q - 1)$$

Esta función indica el número de enteros positivos menores que n que son coprimos con n . Y es fundamental para la clave privada.

Se proporciona el exponente e , que solo es valido si se cumple $\gcd(e, \phi(n)) = 1$. Es decir, e debe ser coprimo con $\phi(n)$. En la implementación esta verificación se realiza mediante el algoritmo de Euclides.

En el cálculo del inverso modular, se utiliza el algoritmo extendido de Euclides para encontrar el valor del exponente privado d tal que

$$ed \equiv 1 \text{ mód } \phi(n)$$

este número constituye la clave privada.

El cifrado de un mensaje M se obtiene mediante $C \equiv M^e \pmod{n}$. La potenciación modular garantiza que el resultado se mantenga entre 0 y $n - 1$. El descifrado se realiza mediante $M \equiv C^d \pmod{n}$. De esta forma el resultado coincide con el mensaje original.

Podemos asegurar que RSA funciona porque el exponente privado d es el inverso modular del exponente público e . Esto significa que

$$ed \equiv 1 \pmod{\phi(n)}$$

De acuerdo con el teorema de Euler, si M es coprimo con n , entonces

$$M^{\phi(n)} \equiv 1 \pmod{n}$$

Como consecuencia,

$$(C)^d = (M^e)^d = M^{ed} \equiv M \pmod{n}$$

lo que garantiza que el descifrado del mensaje cifrado C nos devuelve el mensaje exactamente el mensaje original M .

4. MPC básico: calcular un promedio sin mostrar los datos

Cada nota se divide en tres partes aleatorias. Para una nota x , se generan valores aleatorios s_1 y s_2 . La tercera parte se calcula como $s_3 \equiv x - s_1 - s_2 \pmod{M}$, donde M es un número muy grande. De esta forma se garantiza que

$$x \equiv s_1 + s_2 + s_3 \pmod{M}$$

Cada servidor recibe una de las partes aleatorias. Por ejemplo, si una nota es 40, podría dividirse como

Nota	Servidor 1	Servidor 2	Servidor 3
40	132456	654321	213269

El servidor 1 conoce únicamente 132456. Ese valor fue generado aleatoriamente y no tiene relación con la nota original, por lo que no es posible deducir la nota original. Lo mismo ocurre con los otros servidores.

Para la reconstrucción de la suma, cada servidor envía la suma de todas sus partes. Entonces la suma total de las notas se obtiene mediante

$$Suma \equiv S_1 + S_2 + S_3 \pmod{M}$$

Finalmente, el promedio se calcula de la forma habitual, dividiendo la suma total entre el número de notas. De esta forma, es posible calcular el promedio sin revelar las notas individuales.

5. MPC básico: calcular un promedio sin mostrar los datos

Una red de transporte puede modelarse mediante un grafo ponderado $G = (V, E)$, donde V es el conjunto de nodos y E es el conjunto de aristas. Cada arista $(u, v) \in E$ tiene un peso asociado $w(u, v)$ que representa el costo o la distancia entre los nodos u y v . El objetivo es encontrar el camino más corto desde un nodo de origen s hasta un nodo de destino t . Si un camino está formado

por las aristas $(u_1, u_2), (u_2, u_3), \dots, (u_{k-1}, u_k)$, entonces el costo total del camino es la suma de los pesos de las aristas que lo componen:

$$d(u_1, u_k) = \sum_{i=0}^{k-1} w(u_i, u_{i+1})$$

El algoritmo busca minimizar esta suma para encontrar el camino más corto entre los nodos de origen y destino.

Dijkstra mantiene para cada vértice una estimación de la distancia más corta desde el nodo de origen. Inicialmente, la distancia al nodo de origen es cero y la distancia a todos los demás nodos es infinita. A medida que se exploran los nodos, se actualizan las distancias si se encuentra un camino más corto. Para esto el algoritmo realiza dos pasos:

1. Seleccionar el nodo con la distancia más corta conocida y marcarlo como visitado.
2. Actualizar las distancias de los nodos adyacentes al nodo seleccionado, si se encuentra un camino más corto a través de él.

Si desde un vertice u existe una arista hacia un vertice v , se verifica si el camino

$$d(u) + w(u, v)$$

es mejor que la distancia actualmente conocida para v . Si es así, se actualiza la distancia de v y se registra que el camino más corto hacia v pasa por u :

$$d(v) = d(u) + w(u, v)$$

Al mismo tiempo se guarda que el predecesor de v es u , lo que posteriormente permite reconstruir la ruta completa.

El algoritmo supone que, una vez seleccionado el vértice con la menor distancia conocida, esa distancia ya es la mínima posible y nunca volverá a mejorar. Esta propiedad únicamente se cumple cuando todas las aristas poseen pesos no negativos. Si existieran pesos negativos, podría aparecer posteriormente un camino más corto hacia un vértice que ya había sido considerado definitivo, invalidando el algoritmo. Por esta razón Dijkstra únicamente garantiza resultados correctos en grafos cuyos pesos sean mayores o iguales que cero.

Además, un camino es optimo cuando su costo total es menor o igual que el de cualquier otro camino entre el mismo origen y destino. Es decir, si P^* es un camino encontrado por dijkstra, entonces para cualquier otro camino P se cumple

$$Costo(P^*) \leq Costo(P)$$

Esto garantiza que la distancia obtenida corresponde a la ruta más corta disponible en el grafo.

6. Cierre de una estación: medir el impacto en la red

Para este problema, se busca medir el impacto de cerrar una estación en la red de transporte o las conexiones entre estaciones. Para la modelación del problema, se basa en el algoritmo de Dijkstra y con las mismas condiciones que se expresaron en la sección anterior.

6.1. Cierre de una estación

Cerrar una estación equivale a eliminar un vértice del grafo junto con todas las conexiones incidentes. Si el vertice eliminado es v , el grafo resultante queda definido como $G' = (V \setminus \{v\}, E')$, donde E' es el conjunto de aristas que no están conectadas al vértice eliminado. Posteriormente, se vuelven a calcular las rutas más cortas desde el nodo de origen hasta todos los demás nodos en el grafo modificado G' .

6.2. Cierre de una conexión

Si únicamente se elimina una conexión entre dos estaciones u y v , el conjunto de vértices permanece igual y únicamente desaparece una arista: $G' = (V, E \setminus \{(u, v)\})$. Nuevamente se ejecuta Dijkstra para calcular las nuevas ruta minimas. Finalmente, las consecuencias son presentadas en la medición del impacto.

6.3. Medición del impacto

Para cada par origen-destino se comparan las distancias antes y después del cierre de la estación o conexión. Si tenemos

- d_{antes} es la distancia original
- $d_{despues}$ es la nueva distancia

el aumento producido por el cierre es la diferencia $\Delta d = d_{despues} - d_{antes}$

Se presentan tres posibles resultados:

- Si $\Delta d = 0$, significa que el cierre no afectó la ruta entre el par de estaciones.
- Si $\Delta d > 0$, indica que la ruta se volvió más larga debido al cierre.
- Si $\Delta d = \infty$, significa que no existe un camino entre las estaciones después del cierre, lo que indica una desconexión en la red.

Si tomamos como ejemplo el grafo de la figura

7. Coloreo de grafos: organizar exámenes sin choques

Se debe evitar que dos materias que comparten estudiantes tengan exámenes al mismo tiempo. Esto se puede modelar mediante un coloreo de grafos, donde cada color representa una franja horaria. Esto se mantiene bajo la misma estructura de grafo ponderado $G = (V, E)$, donde V es el conjunto de materias y E es el conjunto de aristas que representan conflictos entre materias. Por ejemplo, si $(A, B) \in E$, entonces las materias A y B tienen estudiantes en común, por lo que deben recibir colores diferentes.

El objetivo es encontrar una función de coloreo

$$C : V \rightarrow \{0, 1, 2, \dots, k\}$$

donde cada vertice recibe un color, cumpliendo la condición de $C(u) \neq C(v)$ para todo par de vértices adyacentes $(u, v) \in E$.

Además, se busca minimizar el número de colores utilizados. Esta cantidad mínima de colores se conoce como el número cromático del grafo $\chi(G)$. Encontrar exactamente este número es un problema complejo para grafos grandes, por lo que en este caso se utiliza un algoritmo que obtiene una solución valida, aunque no necesariamente óptima. La estrategia consiste en:

1. Se recorren los vértices del grafo en un orden determinado.
2. Para cada vértice se revisan los colores utilizados por sus vecinos ya coloreados.
3. Se asigna el primer color disponible que no genere conflicto.

Matemáticamente, para un vértice v , se define el conjunto de colores usados por sus vecinos:

$$S(v) = \{C(u) | u \in N(v)\}$$

donde $N(v)$ representa el conjunto de vecinos de v . Luego se asigna el menor color entero que no esté siendo utilizado por sus vecinos.

8. Tablas de verdad y circuitos lógicos

Para probar las posibilidades de una expresión lógica, se construye una tabla de verdad que permiten revisar todas las combinaciones posibles de valores de entrada y sus correspondientes resultados de salida. Para este problema se consideran las variables booleanas A , B , C y D , que pueden tomar valores de 0 (falso) o 1 (verdadero). Además, sobre estas variables se aplican operaciones lógicas como AND ($\wedge$), OR ($\vee$), NOT ($\neg$) y XOR ($\oplus$).

Al tener n variables booleanas, la tabla de verdad tendrá 2^n filas, representando todas las combinaciones posibles de valores de entrada. Por tanto, para 4 variables, la tabla tendrá $2^4 = 16$ filas. Finalmente, se evalúa la expresión lógica para cada combinación de valores de entrada y se registra el resultado en la columna correspondiente de la tabla.

Matemáticamente, cada expresión lógica puede representarse mediante una función booleana $f : \{0, 1\}^4 \rightarrow \{0, 1\}$. Es decir, recibe 4 expresiones booleanas como entrada y produce un resultado booleano como salida. Esto es super importante para el diseño de circuitos digitales, ya que en la electrónica digital, las variables booleanas representan señales eléctricas binarias.

- 0 corresponde a un nivel bajo de voltaje (falso)
- 1 corresponde a un nivel alto de voltaje (verdadero)

De igual manera cada operador lógico tiene una representación física mediante compuertas lógicas. Con esto se tiene que la salida del circuito para cada combinación de entradas es exactamente la misma que la salida de la función booleana correspondiente. Por esta razón, la tabla de verdad constituye una descripción completa del comportamiento del circuito lógico: para cualquier combinación posible de entradas, indica cuál será el valor de salida.

9. Simplificación booleana: hacer un circuito más barato

En lógica digital, la simplificación booleana es un proceso que busca reducir la complejidad de una expresión lógica sin alterar su funcionalidad. Esto se logra mediante el uso de leyes y teoremas del álgebra de Boole, como la ley de De Morgan, la ley de absorción y la ley distributiva. El objetivo es obtener una expresión equivalente que utilice menos términos o compuertas lógicas, lo que se traduce en un circuito más eficiente y económico.

Un mintermino es un producto que incluye todas las variables de entrada de la función, cada una en su forma directa o negada, y que vale 1 para exactamente una combinación de entradas. Para una función de 3 variables (A , B , C), el mintermino número 5 corresponde a la fila donde $A=1$, $B=0$, $C=1$ (porque 5 en binario de 3 bits es '101'), y se escribe como el término $A \cdot B' \cdot C$. De

esta manera, una función puede describirse simplemente indicando los minterminos donde su salida vale 1. Por ejemplo,

$$F(A, B, C) = \sum m(1, 3, 5, 7)$$

significa que la función es verdadera únicamente para los minterminos 1, 3, 5 y 7.

Se usa la suma de productos (SOP) para representar la función booleana como una suma de minterminos. Esto consiste en realizar la operación OR entre todos los minterminos que hacen que la función sea verdadera, por ejemplo:

$$F(A, B, C) = \bar{A}\bar{B}C + \bar{A}BC + A\bar{B}C + ABC$$

Aunque esta representación siempre es correcta, generalmente contiene términos repetitivos y puede simplificarse. En el ejemplo anterior, todos los términos contienen la variable C, por lo que la función simplificada es:

$$F(A, B, C) = C$$

Esta nueva expresión produce exactamente el mismo resultado utilizando una sola variable, reduciendo considerablemente la complejidad del circuito.

Para esta solución se usa una versión pequeña de Quine-McCluskey. Este algoritmo simplifica funciones booleanas agrupando minterminos que difieren únicamente en una variable. Por ejemplo $\bar{A}\bar{B}C$ y $\bar{A}BC$ difieren únicamente en la variable B . Al combinarse, se obtiene un término más simple $\bar{A}C$. Este proceso se repite hasta que no se puedan combinar más términos.

Un implicante primo es un término (posiblemente con variables eliminadas) que no puede combinarse más con ningún otro. Un implicante primo es esencial cuando es la única opción disponible para cubrir algún mintermino de la función; estos siempre deben incluirse en la expresión final. Los minterminos que ningún esencial cubra se resuelven eligiendo, entre los implicantes restantes, el que cubra más minterminos pendientes.

Los valores "no importa" (don't-cares) son combinaciones de entrada donde no interesa el resultado; se usan durante la combinación de términos para lograr implicantes más grandes, pero no se exige que la expresión final los cubra.